

33. offline forecast 取得 CLI

solar_ws0129-main

2026-07-29

対象

項目	内容
ファイル	scripts/fetch_weather_forecast.py
種別	Python
区分	offline tool
役割	profile に基づき計画用 weather forecast CSV を取得・保存する CLI スクリプト。
起動文脈	forecast action で直接呼ばれる。

このファイルを読む理由

profile に基づき計画用 weather forecast CSV を取得・保存する CLI スクリプト。

- live node 版より単発実行向け。

呼び出し関係

どこから呼ばれるか

- scripts/solar_control.sh

次に読むと理解が進むファイル

- mpc_solarcar/weather_utils.py
- mpc_solarcar/solar_profile.py

同一リポジトリ内の主要依存

- mpc_solarcar/solar_profile.py
- mpc_solarcar/weather_utils.py

ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

代表コード断片

```
from mpc_solarcar.weather_utils import fetch_openmeteo_forecast, write_forecast_csv

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--profile_yaml', required=True)
    ap.add_argument('--latitude', type=float, default=None)
    ap.add_argument('--longitude', type=float, default=None)
    ap.add_argument('--out_csv', default='')
    ap.add_argument('--forecast_days', type=int, default=None)
    ap.add_argument('--step_minutes', type=int, default=None)
    ap.add_argument('--timezone_name', default=None)
    ap.add_argument('--tcell_gain', type=float, default=None)
    args = ap.parse_args()

    profile_path, cfg = load_profile(args.profile_yaml)
    runtime_cfg = get_section(cfg, 'runtime')
    live_cfg = get_section(cfg, 'live')
    weather_cfg = merged_dict({
        'forecast_days': 3,
        'step_minutes': 10,
        'timezone_name': 'Australia/Darwin',
```

主要構造の読み取り

主要関数は main。CLI 引数宣言は 8 件。

主要クラス・関数

行	種別	名前	補足
17	function	main	

ファイルを上から読んだときの定義順

行	内容
9	ROOT に Path(__file__).resolve().parents[1] の結果を代入する。
10	条件 str(ROOT) not in sys.path を判定し、真なら内部処理を行う。
17	関数 main を定義する。
82	条件 __name__ == '__main__' を判定し、真なら内部処理を行う。

import 群の詳細

行	import 文	このファイルで読む理由
2	import argparse	CLI 引数を宣言し、実行時パラメータを外から受け取るため。このファイル内での主な使用位置は L18。
3	import os	パス、環境変数、プロセス外部状態を扱うため。このファイル内での主な使用位置は L54, L77。

4	<code>importsys</code>	sys モジュールを利用するため。このファイル内での主な使用位置は L10, L11。
5	<code>frompathlibimportPath</code>	ファイルやディレクトリを安全に扱うため。このファイル内での主な使用位置は L9。
7	<code>importpandasaspd</code>	CSV や時刻列の読込・整形・再サンプリングに使うため。このファイル内での主な使用位置は L55。
13	<code>frommpc_solarcar.solar_profileimportget_path, get_section, load_profile, merged_dict</code>	profile YAML 読込と検証から <code>get_path</code> , <code>get_section</code> , <code>load_profile</code> , <code>merged_dict</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/solar_profile.py</code> 。このファイル内での主な使用位置は L29, L30, L31, L32, L39, L51, L52。
14	<code>frommpc_solarcar.weather_utilsimportfetch_openmeteo_forecast, write_forecast_csv</code>	<code>weather_utils.py</code> から <code>fetch_openmeteo_forecast</code> , <code>write_forecast_csv</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/weather_utils.py</code> 。このファイル内での主な使用位置は L69, L78。

関数・クラスを上から順に解説

L17 関数 main

- 定義: `main()`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ArgumentParser`, `abspath`, `add_argument`, `dirname`, `exists`, `fetch_openmeteo_forecast`, `float`, `get`, `get_path`, `get_section`, `int`, `len`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。

1. `ap` に `argparse.ArgumentParser()` の結果を代入する。
2. `ap.add_argument(...)` を実行する。
3. `ap.add_argument(...)` を実行する。
4. `ap.add_argument(...)` を実行する。
5. `ap.add_argument(...)` を実行する。
6. `ap.add_argument(...)` を実行する。
7. `ap.add_argument(...)` を実行する。
8. `ap.add_argument(...)` を実行する。
9. `ap.add_argument(...)` を実行する。
10. `args` に `ap.parse_args()` の結果を代入する。

CLI 引数

行	引数
19	<code>--profile_yaml</code>
20	<code>--latitude</code>

```
21      --longitude
22      --out_csv
23      --forecast_days
24      --step_minutes
25      --timezone_name
26      --tcell_gain
```

処理の流れ

1. CLI 引数を解釈し、`main()` から処理を起動する。

このファイルの位置づけ

この資料群では、`scripts/fetch_weather_forecast.py` を **offline tool** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。